Homework 4: State tracking and sliding window

Guideline

Homework chatbot. Open the homework Gem here. Use this Gem when working through the AI-related sections. The bot is already curated with the right answers and hints to provide the right level of guidance to help you learn.

Showing your honest attempt can only help. There is no advantage in cheating. I give 100% credit on attempts, even if they are incorrect. Cheating on homework will also only hurt you. If you cheat, I won't be able to give you the proper feedback to help you learn, nor adjust the difficulty of the quizzes. You will also likely fail on the in-person mastery quizzes, which test concepts tackled in the homework.

Collaboration: Only after solo attempt. You may collaborate and use external resources, but (1) you are encouraged to do the homework **alone first** and (2) you must **attribute** these external resources when you do use them.

AI: Follow guideline. The problems will be very clear on when you are allowed to use AI. Please follow them. Here is why: (1) Unregulated usage of AI harms learning (2) Even in the age of AI, your AI-independent ability matters. The **only** AI tool you should use on this homework is the homework-specific Gemini Gem that I have created for you.

Submission. Upload all files to the Homework 4 assignment on Canvas. **9 files total:**

1. **One answer document** containing your answers to every question below. Name it `lastname-hw4.pdf` (or `.txt`). LaTeX isn't required -- plain text, Google Docs → Download as PDF, or Word → Save as PDF are all fine.
2. **Eight AI chat transcript PDFs**, one for each AI-check step. Start a **new chat** in the Gem for each step, so that each transcript covers only that step:
    - `flip-b.pdf`, `flip-e.pdf`
    - `maxproduct-b.pdf`, `maxproduct-e.pdf`
    - `ones-b.pdf`, `ones-d.pdf`
    - `minsubarray-b.pdf`, `minsubarray-d.pdf`

Accepted file formats: `.pdf` for everything; `.txt` is also fine for the answer document. Do **not** paste chat content into your answer document, and do **not** submit a link to a Google Doc -- upload the file itself.

Due: Sunday, October 18, 2026 at 11:59pm. See the syllabus for the late-day policy.

Points: 39 total. Remember, I give full points to every question that was attempted, and 0 only for a question left blank. What matters is that you have **thought** through the problems enough to prepare yourself for the quizzes.

1. Flip string to monotone increasing (10 pts total)

A binary string is **monotone increasing** if it consists of some number of `0`s (possibly none), followed by some number of `1`s (possibly none). So `"00011"` is monotone increasing, and `"010"` is not.

You are given a binary string `s`. In one operation you may flip one character, `0` to `1` or `1` to `0`. Return the **minimum number of flips** to make `s` monotone increasing.

Leetcode link

Here are some example input and output:

#	`s`	answer	why
1	`"00110"`	1	flip the last `0` to get `"00111"`
2	`"010110"`	2	flip to `"000111"`
3	`"00011000"`	2	flip the two `1`s to get `"00000000"`

(a) No-AI-attempt planning

Do **not** use AI for this part. Understand the problem first, then start a stop watch for **30 minutes** and tackle this on your own. Use the full 30 minutes. If you give up before you have a complete answer, that's fine -- an incomplete attempt still earns full credit.

(i) Define the subproblem (1 pt)

Write down, in **one line each**, what your `dp` array(s) mean.

Hint: you may find that a single `dp` array is not enough here. As in lecture, you are allowed to define **several `dp` arrays, each one tracking a different state.**

(ii) Find the recurrence, and argue why it is correct (1 pt)

Express your `dp` value(s) at index `i` in terms of smaller subproblems. Show the exhaustive cases you are taking the min over.

Then, in 2 to 5 sentences, argue why the recurrence is correct.

(b) AI plan check

Use the homework chatbot to check your answers from part (a). Here are some example prompts:

- **Verifying.** "Here is the problem statement (copy paste). Here is my dp definition and recurrence (copy paste). Are these right?"
- **Brainstorming.** "I'm stuck. Can you give me a hint?"
- **Asking for clarification.** "Why isn't one dp array enough here?"

(i) Chat transcript (1 pt)

Turn in a **chat transcript** as a PDF. Name it `flip-b.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(c) Compute the first entries by hand (1 pt)

Using **your own** definition from (a), compute the values of **each** of your arrays for the first three prefixes of `s = "010110"` -- that is, for `"0"`, then `"01"`, then `"010"`.

(d) Code (1 pt)

Start a stop watch for **30 minutes** and code this on leetcode without using AI. Imagine that this is you having to write code on the whiteboard for an interview, and you are under time pressure.

If you have exceeded 30 minutes, you **may** use the homework AI to help you succeed and get a successful leetcode submission.

Turn in your final code, pasted into your answer document. Also paste your Leetcode submission URL showing a successful submission.

(e) AI code check

Copy paste your code to the homework gem and ask for AI review. Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Can you evaluate for correctness? Q2. Can you help improve my code style?"

(i) Chat transcript (1 pt)

Turn in the chat transcript PDF. Name it `flip-e.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(f) Runtime (1 pt)

What is the big-O runtime of your solution, and why?

(g) Space complexity (1 pt)

What is the big-O space usage? Can it be reduced? If so, to what, and what makes that possible?

2. Maximum product subarray (10 pts total)

Given an integer array `nums`, find a **contiguous** subarray that has the largest **product**, and return that product. The subarray must be non-empty.

Leetcode link

Here are some example input and output

#	`nums`	answer	why
1	`[2, 3, -2, 4]`	6	the subarray `[2, 3]`
2	`[-2, 0, -1]`	0	the subarray `[0]`; note the answer is not `2`

(a) No-AI-attempt planning

Do **not** use AI for this part. Understand the problem first, then start a stop watch for **30 minutes** and tackle this on your own. Use the full 30 minutes. If you give up before you have a complete answer, that's fine -- an incomplete attempt still earns full credit.

(i) Define the subproblem (1 pt)

Write down, in **one line each**, what your `dp` array(s) mean.

Hint: you may find that a single `dp` array is not enough here. As in lecture, you are allowed to define **several `dp` arrays, each one tracking a different state.** Do you ever need to care about values that are not max / min?

(ii) Find the recurrence, and argue why it is correct (1 pt)

Express your `dp` value(s) at index `i` in terms of smaller subproblems. Show the exhaustive cases you are taking the max (or min) over.

Then, in 2 to 5 sentences, argue why the recurrence is correct. Be explicit about what happens when `nums[i]` is negative.

(b) AI plan check

Use the homework chatbot to check your answers from part (a). Here are some example prompts:

- **Verifying.** "Here is the problem statement (copy paste). Here is my dp definition and recurrence (copy paste). Are these right?"
- **Brainstorming.** "I'm stuck. Can you give me a hint?"
- **Asking for clarification.** "Why do I need to track a minimum when the question asks for a maximum?"

(i) Chat transcript (1 pt)

Turn in a **chat transcript** as a PDF. Name it `maxproduct-b.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(c) Compute the first entries by hand (1 pt)

Using **your own** definition from (a), compute the values of **each** of your arrays for subarrays that end exactly at the **1st**, **2nd**, and **3rd** element of `nums = [2, 3, -2, 4]` -- that is, ending at `2`, then at `3`, then at `-2`.

(d) Code (1 pt)

Start a stop watch for **30 minutes** and code this on leetcode without using AI. Imagine that this is you having to write code on the whiteboard for an interview, and you are under time pressure.

If you have exceeded 30 minutes, you **may** use the homework AI to help you succeed and get a successful leetcode submission.

Turn in your final code, pasted into your answer document. Also paste your Leetcode submission URL showing a successful submission.

(e) AI code check

Copy paste your code to the homework gem and ask for AI review. Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Can you evaluate for correctness? Q2. Can you help improve my code style?"

(i) Chat transcript (1 pt)

Turn in the chat transcript PDF. Name it `maxproduct-e.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(f) Runtime (1 pt)

What is the big-O runtime of your solution, and why?

(g) Space complexity (1 pt)

What is the big-O space usage? Can it be reduced? If so, to what, and what makes that possible?

3. Max consecutive ones III (9 pts total)

You are given a binary array `nums` and an integer `k`. You may flip **at most** `k` of the zeros into ones. Return the length of the **longest run of consecutive ones** you can produce.

Leetcode link

Our running example is `nums = [1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 0]` with `k = 2`, whose answer is **6**. Flip the zeros at index 4 and index 5 to get `[1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0]`, whose longest run of ones is the stretch from index 4 to index 9.

The other Leetcode example:

#	`nums`	`k`	answer
1	`[1,1,1,0,0,0,1,1,1,1,0]`	2	6
2	`[0,0,1,1,0,0,1,1,1,0,1,1,0,0,0,1,1,1,1]`	3	10

(a) No-AI-attempt planning

Do **not** use AI for this part. Work through the first two steps of the sliding window cookbook on your own.

(i) Step 1a. What does "good" mean? (1 pt)

We are looking for the **longest good window**. In one line, what does *good* mean for this problem?

Careful: your answer should be a property of a window that you can check without doing any flipping. Nothing in your final code should ever modify `nums`.

(ii) Step 1b. What variables? (1 pt)

What variable, or variables, would you keep so that you can check your "good" property in O(1) as the window changes? Say what each one holds, and how it updates when the right end moves in and when the left end moves out.

(iii) Step 2. Argue the sliding window property (1 pt)

Write 3 to 5 sentences. Reason about it like this: suppose you already know `f(r)`, the **leftmost** index such that the window from `f(r)` to `r` is still good. Now move the right end to `r+1`. Can `f(r+1)` ever be **less than** `f(r)`? Argue why or why not.

(b) AI plan check

Use the homework chatbot to check your answers from part (a). Here are some example prompts:

- **Verifying.** "Here is the problem statement (copy paste). Here is my definition of a good window, my variables, and my argument (copy paste). Are these right?"
- **Brainstorming.** "I'm stuck on what to track. Can you give me a hint?"
- **Asking for clarification.** "Why can the left end never move backwards here?"

(i) Chat transcript (1 pt)

Turn in a **chat transcript** as a PDF. Name it `ones-b.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(c) Step 3. Code (1 pt)

Start a stop watch for **30 minutes** and code this on leetcode without using AI. Imagine that this is you having to write code on the whiteboard for an interview, and you are under time pressure.

If you have exceeded 30 minutes, you **may** use the homework AI to help you succeed and get a successful leetcode submission.

Turn in your final code, pasted into your answer document. Also paste your Leetcode submission URL showing a successful submission.

(d) AI code check

Copy paste your code to the homework gem and ask for AI review. Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Can you evaluate for correctness? Q2. Can you help improve my code style?"

(i) Chat transcript (1 pt)

Turn in the chat transcript PDF. Name it `ones-d.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(e) Runtime (1 pt)

What is the big-O runtime of your solution, and **why**? Your answer should explain why the nested loop does **not** make this quadratic.

4. Minimum size subarray sum (10 pts total)

We discussed this problem in lecture, and now you will get to implement the solution in full!

Given an array of **positive** integers `nums` and a positive integer `target`, return the **minimal length** of a contiguous subarray whose sum is greater than or equal to `target`. If there is no such subarray, return `0`.

Leetcode link

Here are some example input and output:

#	`target`	`nums`	answer	why
1	7	`[2,3,1,2,4,3]`	2	the subarray `[4, 3]`
2	4	`[1,4,4]`	1	the subarray `[4]`
3	11	`[1,1,1,1,1,1,1]`	0	no subarray reaches 11

(a) No-AI-attempt planning

Do **not** use AI for this part. Work through the first two steps of the sliding window cookbook on your own.

This one is a **shortest** window problem, so before anything else you must flip it into the "longest good window" shape we used in lecture. Do that first -- the rest of part (a) is about the flipped version.

(i) Step 1a. What does "good" mean? (1 pt)

Restate this as a **longest good window** problem, and say in one line what *good* means in your restatement.

Then say, in one line, how you recover the answer to the original question once you know the longest good window ending at each `r`.

(ii) Step 1b. What variables? (1 pt)

What variable, or variables, would you keep so that you can check your "good" property in O(1) as the window changes? Say what each one holds, and how it updates when the right end moves in and when the left end moves out.

(iii) Step 2. Argue the sliding window property (1 pt)

Write 3 to 5 sentences. Same reasoning as before: suppose you already know `f(r)`, the **leftmost** index such that the window from `f(r)` to `r` is still good. Now move the right end to `r+1`. Can `f(r+1)` ever be **less than** `f(r)`? Argue why or why not. Where exactly does your argument use the fact that all the numbers are positive?

(b) AI plan check

Use the homework chatbot to check your answers from part (a). Here are some example prompts:

- **Verifying.** "Here is the problem statement (copy paste). Here is how I flipped it into a longest-window problem, plus my variables and my argument (copy paste). Are these right?"
- **Brainstorming.** "I don't see how to turn a shortest-window question into a longest-window question. Can you give me a hint?"
- **Asking for clarification.** "Once I have the longest window under the target, how do I get back the answer to the original question?"

(i) Chat transcript (1 pt)

Turn in a **chat transcript** as a PDF. Name it `minsubarray-b.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(c) Step 3. Code (1 pt)

Start a stop watch for **30 minutes** and code this on leetcode without using AI. Imagine that this is you having to write code on the whiteboard for an interview, and you are under time pressure.

If you have exceeded 30 minutes, you **may** use the homework AI to help you succeed and get a successful leetcode submission.

Turn in your final code, pasted into your answer document. Also paste your Leetcode submission URL showing a successful submission.

(d) AI code check

Copy paste your code to the homework gem and ask for AI review. Here is an example prompt: "Here is the problem statement (copy) and here is my code (copy). Q1. Can you evaluate for correctness? Q2. Can you help improve my code style?"

(i) Chat transcript (1 pt)

Turn in the chat transcript PDF. Name it `minsubarray-d.pdf`. Do **not** copy paste the chat content into your answer document.

(ii) Human reconciliation (1 pt)

Summarize in your answer document. Any corrections you made? Any new learnings? You can also say "Nope, I got everything right."

(e) Runtime (1 pt)

What is the big-O runtime of your solution, and **why**?

(f) What if the array had negative numbers? (1 pt)

Leetcode guarantees that every element of `nums` is positive. Suppose we drop that guarantee and allow negative numbers, keeping everything else the same. Can we still use sliding window?

Provide a **concrete counterexample**. Give a small array containing at least one negative number, and a target, where the sliding window code you wrote in (c) returns the **wrong** answer.

Then, **argue** in 3 to 5 sentences, for what exactly broke. Use the vocabulary from lecture: what happens to the pattern of good and bad windows as you slide `l` from left to right, and why does that make "the leftmost good `l`" stop being a meaningful thing to chase?